

AI6103 Homework Assignment

Effects of hyperparameters on EfficientNet-B0 and Food-11 Dataset

Nikunj Madan

G2504045E

College of Computing and Data Science
Nanyang Technological University

Abstract

This assignment analyzes the effects of hyperparameters like learning rate, learning rate schedule, weight decay and data augmentation on deep neural networks. Through this assignment we understand how a model is optimized with the help of learning rate and learning rate schedule. Furthermore we also understand how a model's complexity is controlled with the help of regularization techniques like weight decay and mixup data augmentation.

The experiments are performed on the EfficientNet-B0 network on the Food-11 dataset. Throughout the experiments a SGD optimizer with momentum of 0.9 and batch size of 128 is used. Each experiment provides us with the optimal hyperparameters which yielded an accuracy of 81.63% on unseen test data.

Introduction

This report discusses the effects of optimization techniques like learning rate and learning rate schedule, and regularization techniques like weight decay and data augmentation. Optimization techniques aim to minimize the error and maximize the performance of the model. Whereas regularization techniques help to prevent overfitting by reducing the model's complexity thereby improving generalization.

For evaluation of these hyperparameters a EfficientNet-B0 network (a lightweight convolution neural network) is trained on the Food-11 dataset. Throughout the ablation testing a SGD optimizer with momentum of 0.9 is used. The results of the ablation testing are evaluated on the validation dataset with the help of loss and accuracy curves.

Through these iterative experiments, at each stage we find the optimal hyperparameter that improves the model's generalization and prediction making capabilities. Firstly we find the hyperparameters for optimization techniques followed by the regularization techniques. The insights gained at from each experiment are highlighted in this report along with experiment specific trends and tables to support the insights gained. Finally, after conducting all of the mentioned experiments optimal hyperparameters are found and used to generate predictions on unseen test dataset.

Data Preprocessing

The Food-11 dataset is used to train the EfficientNet-B0 network. The Food-11 dataset consists of 16,643 images which

Channel	Mean	Standard Deviation
Red	0.5548	0.2653
Green	0.4508	0.2677
Blue	0.3435	0.2749

Table 1: Mean and standard deviation for each color channel

```
print(f"Number of training data images: {len(os.listdir(training_data))}")
print(f"Number of validation data images: {len(os.listdir(val_data))}")
print(f"Number of test data images: {len(os.listdir(test_data))}")

Number of training data images: 9866
Number of validation data images: 3430
Number of test data images: 3347

# calculating mean and std for each color channel
mean = torch.zeros(3)
mean_sq = torch.zeros(3)
num_pixels = 0

for images, _ in train_loader:
    batch_size, c, h, w = images.shape
    num_pixels += batch_size * h * w
    mean += images.sum(dim=[0, 2, 3])
    mean_sq += (images ** 2).sum(dim=[0, 2, 3])

mean /= num_pixels
std = torch.sqrt(mean_sq / num_pixels - mean ** 2)

print(f"Channel's mean: Red: {mean[0]:.4f}, Green: {mean[1]:.4f}, Blue: {mean[2]:.4f}")
print(f"Channel's standard deviation: Red: {std[0]:.4f}, Green: {std[1]:.4f}, Blue: {std[2]:.4f}")

Channel's mean: Red: 0.5548, Green: 0.4508, Blue: 0.3435
Channel's standard deviation: Red: 0.2653, Green: 0.2677, Blue: 0.2749

train_transform = transforms.Compose([
    transforms.Resize((100, 100)),
    transforms.RandomHorizontalFlip(p = 0.5),
    transforms.RandomCrop(100, padding = 12, padding_mode = "constant", fill = 0),
    transforms.ToTensor(),
    transforms.Normalize(mean = mean.tolist(), std = std.tolist())
])

val_transform = transforms.Compose([
    transforms.Resize((100, 100)),
    transforms.ToTensor(),
    transforms.Normalize(mean = mean.tolist(), std = std.tolist())
])
```

Figure 1: Data pre-processing code snippet

are roughly divided in the ratio of 6:2:2 for training (9,866), validation (3,430) and test (3,347) dataset respectively.

In the data pre-processing pipeline, the images are first resized to a resolution of 100 x 100. To which the mean and standard deviation of each color channel in the training set is computed. These computed values (in Table 1) are then used for normalizing the data. This resizing and normalizing is applied to both the training and validation set. Furthermore, additional data augmentation methods are applied only to the training set.

Data augmentation is a technique used to increase the size of the training data without incorporating any additional/external data. To the training set the following data augmentation techniques are applied:

- **Random Horizontal Flip:** This method flips the image

along its y-axis based on a probability factor. The probability factor used in implementation is 0.5 meaning that there is a 50% chance for the image to be flipped.

- Random Cropping (with Padding):** Firstly, a zero-padding layer is applied to all sides of the image increasing its size to 124 x 124. To this the augmentation technique randomly crops images of size 100 x 100.

Learning Rate

Learning rate is an optimization hyperparameter that determines the step size that is taken to update the model’s weights. This essentially describes the stability and speed of convergence during the training process. To test the effects of learning rate the model was trained for 15 epochs at batch size of 128 with three different learning rates (0.1, 0.025, 0.001).

The training accuracy and loss curve show a steady improvement in the performance over the epochs for learning rate of 0.025 with peak accuracy of 61.83% and lowest loss of 1.1242. The training loss indicates smooth convergence for all learning rates, however the smaller learning rate (0.001) is slowly converging but the step size is too less when compared to the learning rate of 0.025 which achieves a higher accuracy in the same number of epochs.

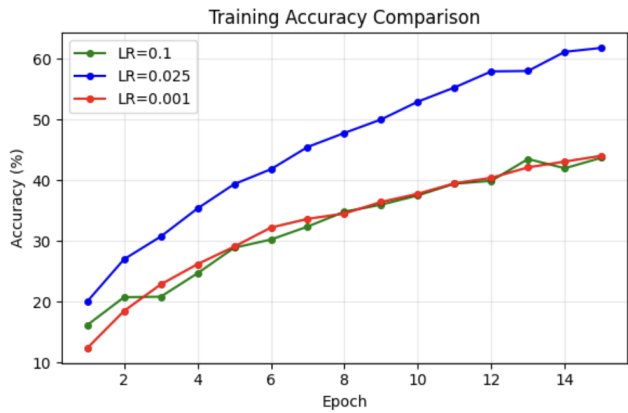


Figure 2: Training accuracy for different learning rate values

Figure 3 and 5 describes the performance of the model on the validation set, The validation accuracy shows that lower learning rates tend to perform better on unseen data in comparison to high learning rates. Moreover, the trend of learning rate of 0.025 achieving the highest accuracy (52.92%) and lowest loss (1.5709) remains the case for validation data as well. On analyzing the validation accuracy of high learning rate a “zig-zag” like pattern is observed indicating instability in training.

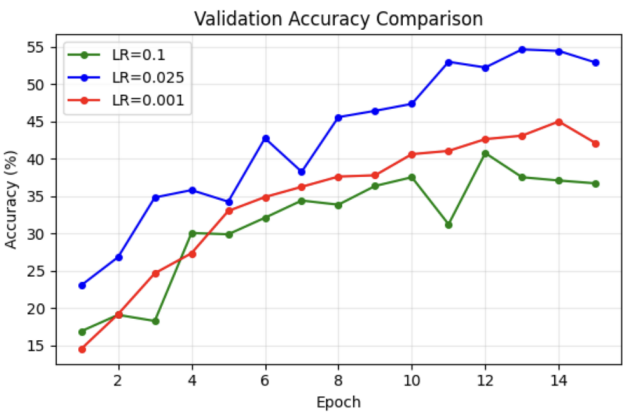


Figure 3: Validation accuracy for different learning rate values

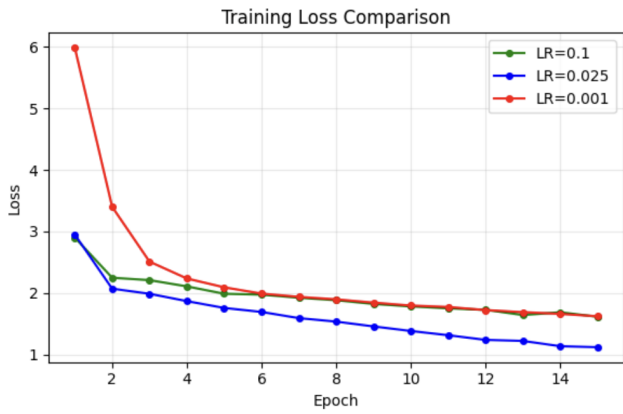


Figure 4: Training loss for different learning rate values

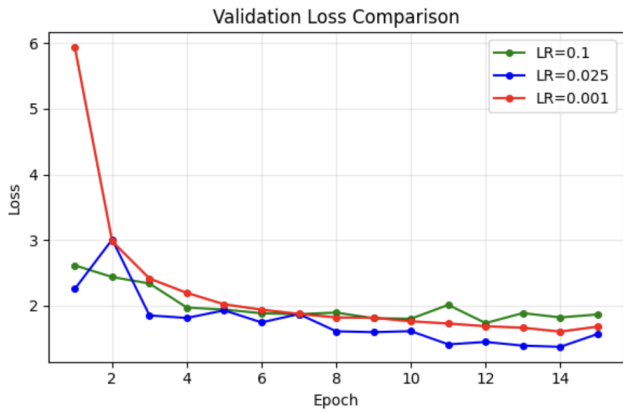


Figure 5: Validation loss for different learning rate values

	LR = 0.1	LR = 0.025	LR = 0.001
Training Accuracy	43.74%	61.83%	44.06%
Training Loss	1.6179	1.1242	1.6251
Validation Accuracy	36.73%	52.92%	42.13%
Validation Loss	1.8691	1.5709	1.6834

Table 2: Learning rate ablation results

Through our observations it can be concluded that a low learning rate (0.001) performs slow convergence while a high learning rate (0.1) is highly unstable. With the final results mentioned in Table 2, we can support and arrive at the conclusion that the optimal learning rate is 0.025 achieving a high training and validation accuracy. In comparison to the other best accuracy values the learning rate of 0.025 achieves 17.77% more training accuracy and 10.79% more validation accuracy.

Learning Rate Schedule

During training the learning rate should be reduced after a certain amount of epochs such that convergence does not plateau. In this section we discuss the effects of using a learning rate schedule like cosine annealing in comparison to using a constant learning rate. Cosine annealing is a popular learning rate scheduler that reduces the learning rate in the form of a cosine curve. It starts with a high initial learning rate and then decreases it rapidly.

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \left(\frac{T_{cur}}{T_{max}} \pi \right) \right) \quad (1)$$

Where η_t represents the learning rate at epoch t , η_{max} denotes the initial (maximum) learning rate, $\eta_{min} = 0$ is the minimum learning rate, and T_{max} indicates the total number of epochs.

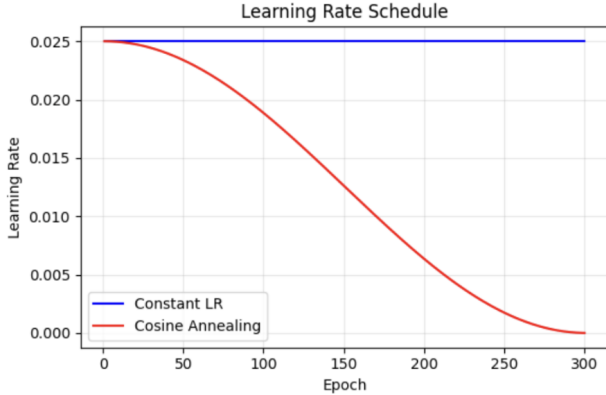


Figure 6: Learning rate trend over training cycle

	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
Constant	0.0252	99.30	1.5171	72.33
Cosine	0.0081	99.80	1.6043	75.13

Table 3: Learning rate schedule ablation results

It is observed that having cosine annealing as a learning rate scheduler yields in achieving a higher accuracy for both training and validation set. It can be concluded that on applying cosine annealing, the validation accuracy improves by 2.8%. Through the trends we can see that the initial high learning rate helps in a faster convergence and the small learning rate in the latter parts of the training cycle helps in smooth generalization towards the minima.



Figure 7: Training accuracy for different learning rate schedules

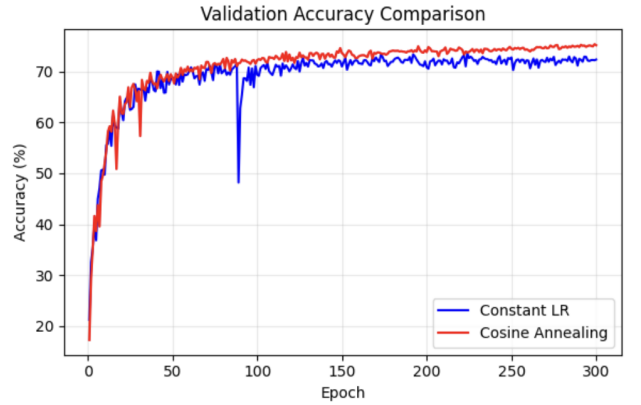


Figure 8: Validation accuracy for different learning rate schedules

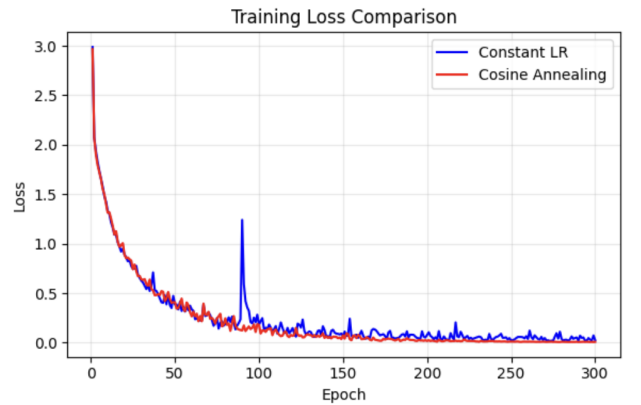


Figure 9: Training loss for different learning rate schedules

On analyzing the training cycle's trends from Figure 7 and 8, cosine annealing schedule significantly improves the generalization. Although both methods have similar training accuracies, cosine annealing achieves a much lower training loss of 0.0081 compared to 0.0252 of constant learning rate. When constant learning rate is used the model is unstable which can be seen from the oscillating nature of the training

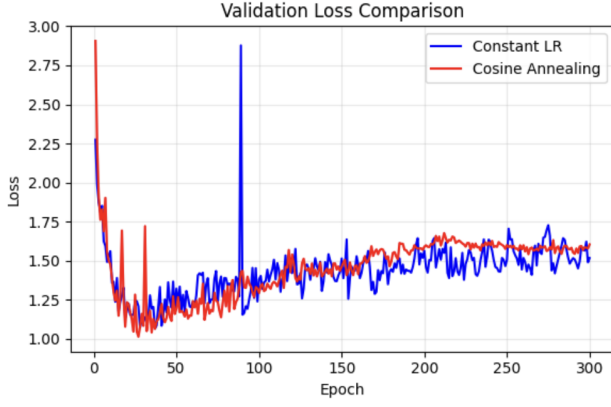


Figure 10: Validation loss for different learning rate schedules

curves. Furthermore, there are some large spikes observed near epoch 90 when using a constant learning rate. Comparatively, the gradual decrease of learning rate through a learning rate scheduler like cosine annealing helps reduce the gradient noise and allows the weights to converge to local minima in a more stable and smoother fashion.

Weight Decay

Weight decay is a regularization method used in neural networks that helps prevent overfitting by penalizing large weights. It is functionally similar to the L_2 regularization used in Ridge Regression (Zhang et al. 2019).

To evaluate this regularization effect in detail, we executed the training pipeline for 300 epochs and compared the behavior of the model using weight decay coefficients $\lambda = 5 \times 10^{-4}$ and 1×10^{-4} .

While both weight decay coefficients perform well and are near-identical in terms of training loss and training accuracy, a major shift is noticeable in the performance on validation set. Higher weight decay value (0.0005) achieved a higher validation accuracy (78.43%) in comparison to 74.81% achieved by smaller weight decay value (0.0001). This 3.62% improvement showed that higher weight decays generalize well when combined with EfficientNet-B0 and cosine annealing. This is also supported by observing the validation loss curve where the overall trajectory of higher weight decay coefficient is converging downwards but the same for a lower weight decay coefficient is upwards and then plateauing after the initial break off.

These trends and observations align with the functionality of weight decay where a higher weight decay coefficient more effectively suppresses the weights of the model. Thereby reducing the model complexity and preventing the model from overfitting.

Weight Decay	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
0.0005	0.0113	99.81	1.0034	78.43
0.0001	0.0058	99.89	1.4537	74.81

Table 4: Weight decay ablation results



Figure 11: Training accuracy for different weight decay values

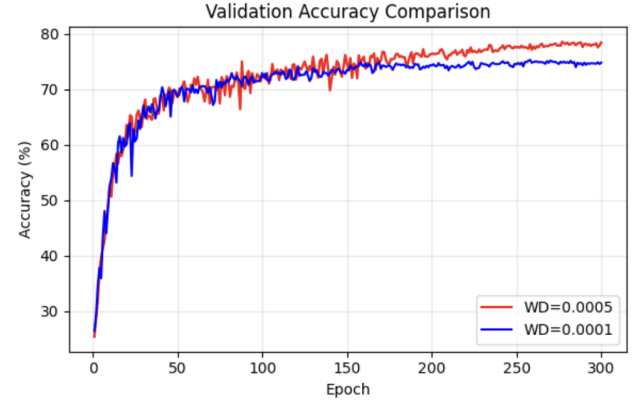


Figure 12: Validation accuracy for different weight decay values

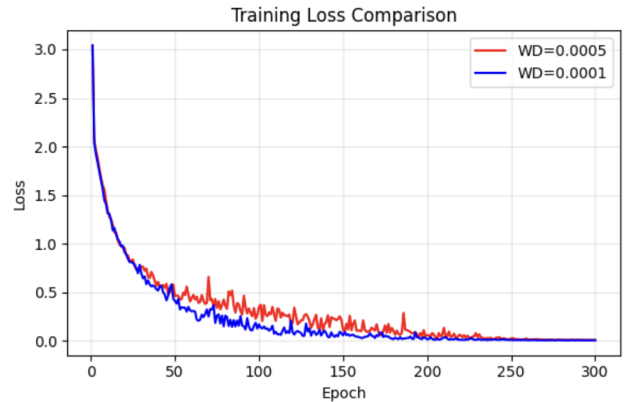


Figure 13: Training loss for different weight decay values

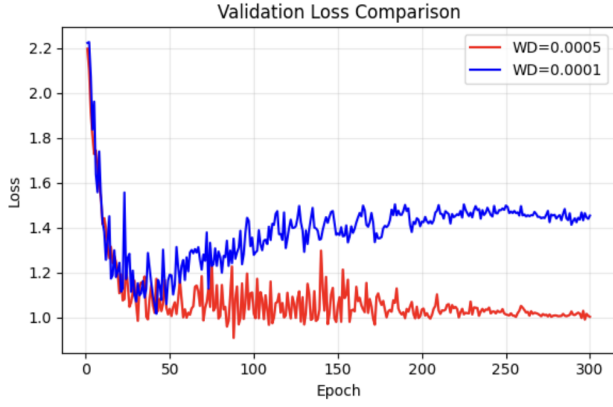


Figure 14: Validation loss for different weight decay values

Data Augmentation

Data augmentation is another regularization technique that helps us to prevent overfitting of the model. This technique increases the amount of training data using only the information that is present in the training data. We have already discussed a few data augmentation techniques like Random Crop and Random Horizontal Flip. In this section we discuss the data augmentation technique called Mixup.

Mixup is a technique that creates new image samples through combination of two image and label pairs. The hyperparameter α dictates the beta distribution through which the mixing factor λ is sampled. Figure 15 shows the probability distribution function which is U-shaped distribution. The small value α indicates that λ values lie near 0 or 1, meaning the new sample is often very close to one of the original sample images. When this α value is increased to be greater than 1, then aggressive sampling is expected.

The probability density function for the interpolation factor is defined by the following equation (DeepAI n.d.):

$$f(\lambda; \alpha) = \frac{\lambda^{\alpha-1}(1-\lambda)^{\alpha-1}}{B(\alpha, \alpha)} \quad (2)$$

Where λ is the interpolation factor, α is the shape parameter (hyperparameter), $B(\alpha, \alpha)$ is the Beta function that acts as a normalization constant.

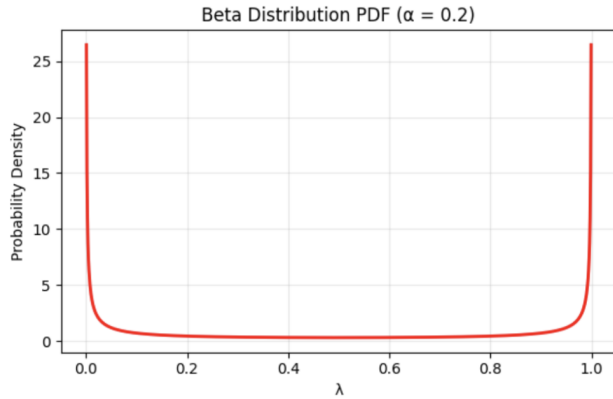


Figure 15: Probability density function associated with the beta distribution at $\alpha = 0.2$

	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
Without Mixup	0.0113	99.81	1.0034	78.43
With Mixup	0.3761	91.22	0.6834	79.83

Table 5: Mixup data augmentation ablation results



Figure 16: Training accuracy with and without mixup data augmentation

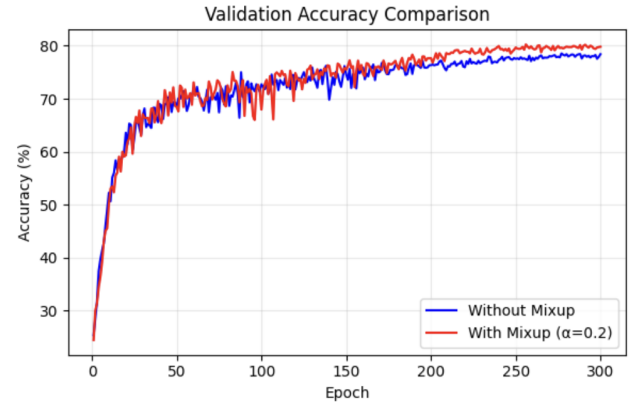


Figure 17: Validation accuracy with and without mixup data augmentation

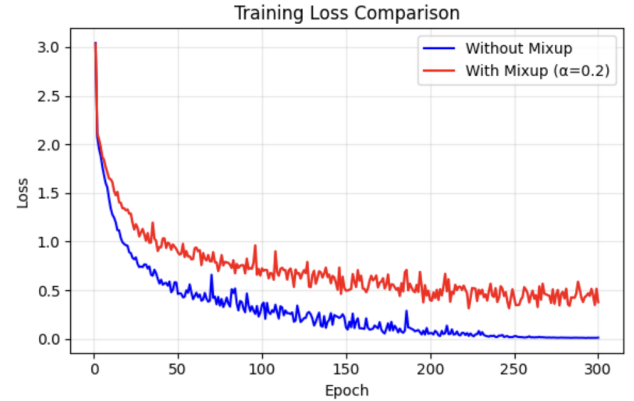


Figure 18: Training loss with and without mixup data augmentation

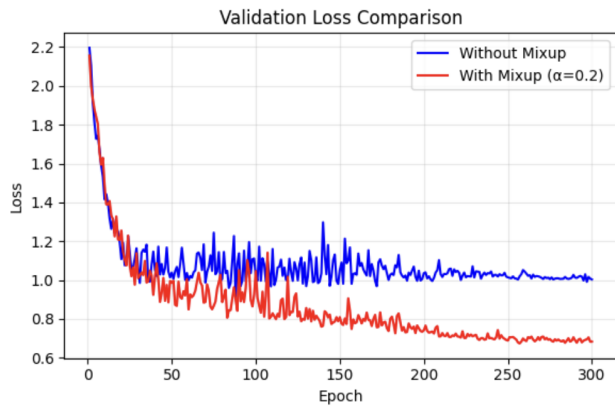


Figure 19: Validation loss with and without mixup data augmentation

With the incorporation of a regularization technique like Mixup data augmentation the training of model becomes more difficult. This is evident by the drop in training accuracy from 99.81% to 91.22% as observed in Figure 16. However, this newly introduced regularization technique introduces the highest validation accuracy (79.83%) and the lowest loss (0.6834) observed so far. These trends show that although the training difficulty has increased, the mixup data augmentation technique helps improve the generalization and predictive capabilities of the model.

Final Evaluation

For the evaluation on the test dataset, the most optimal hyperparameters across the experimental study are chosen. These parameters include:

- Learning rate = 0.025
- Cosine annealing learning rate scheduler
- Weight decay = 0.0005
- Mixup data augmentation

Finally the performance of the EfficientNet-B0 network trained on the Food-11 dataset with the above mentioned configurations for 300 epochs is evaluated.

	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
Validation Set	0.3761	91.22	0.6834	79.83
Test Set	0.3761	91.22	0.6329	81.63

Table 6: Final model performance on validation and test sets

The final evaluation on the unseen test set yielded an accuracy of 81.63% and loss of 0.6329. This performance exceeds the metrics achieved on the validation dataset confirming that the optimal regularization and optimization techniques were used because of the model's improved generalization. If the model had performed poorly in comparison it would show that the model is learning noise and the hyperparameters need to be tuned.

Conclusion

The different experiments conducted showed how different hyperparameters like learning rate, learning rate schedule, weight decay and mixup data augmentation affected the

performance of the EfficientNet-B0 network on the Food-11 dataset. Firstly we analyze how optimization techniques control the stability and speed of convergence in training. Alongside this we also explored regularization techniques and how they help prevent the model from overfitting.

The learning rate investigations showed how a high learning rate leads to unstable training, whereas a low learning rate offers slow convergence. This experiment showed us how a balanced value in between the two cases is optimal for maximizing the performance of the EfficientNet-B0 model.

The learning rate schedule showed the importance of decrementing learning rate over the course of the training cycle. The benefits of gradually decreasing the learning rate with the help of cosine annealing learning rate schedule is observed. This showed us that reducing the learning rate helps reduce the gradient noise and allows the weights to converge to local minima in a more stable and smoother fashion.

The weight decay acts as a L2 regularizer that penalizes large weights. It is observed that a higher weight decay coefficient more effectively suppresses the weights of the model. Thereby reducing the model complexity and preventing the model from overfitting.

Mixup data augmentation method prevents overfitting by increasing the size of the training dataset. This is done by creating new data samples from interpolated combinations of two image and label pairs.

Finally, the best performing hyperparameters from each experiment are chosen and predictions are made on the test dataset. These predictions achieved an accuracy of 81.63% showing that the chosen hyperparameters perform well on an EfficientNet-B0 network trained on the Food-11 dataset.

References

- DeepAI. n.d. Beta Distribution. <https://deepai.org/machine-learning-glossary-and-terms/beta-distribution>. Accessed: 2026-03-23.
- Zhang, G.; Wang, C.; Xu, B.; and Grosse, R. 2019. Three Mechanisms of Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.